

**АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ОРГАНИЗАЦИЯ
ПРОФЕССИОНАЛЬНАЯ ОБРАЗОВАТЕЛЬНАЯ ОРГАНИЗАЦИЯ
"МЕЖДАРОДНЫЙ КОЛЛЕДЖ БИЗНЕСА И ДИЗАЙНА"**

ЛАБОРАТОРНАЯ РАБОТА №3

Тема: Написание более сложных shell-скриптов

выполнила: Слесаревская Иванна

Специальность: информационные
системы и программирование

Группа: ИС-25/9

Цель работы

Научиться писать shell-скрипты, использующие функции, работу со временем, генерацию случайных чисел, работу с аргументами командной строки, файлами, переменными окружения, сигналами и внешними утилитами. Изучить особенности выполнения скриптов командной строки в операционной системе macOS (базирующейся на BSD Unix) и освоить кроссплатформенную адаптацию кода.

Практическая часть

Задание 1. Измерение времени выполнения команды

Был создан скрипт `time_cmd.sh`, адаптированный под macOS. Поскольку стандартная команда `date` на Mac не поддерживает наносекунды (%N), для точного расчёта интервала времени в миллисекундах был задействован встроенный вызов Python.

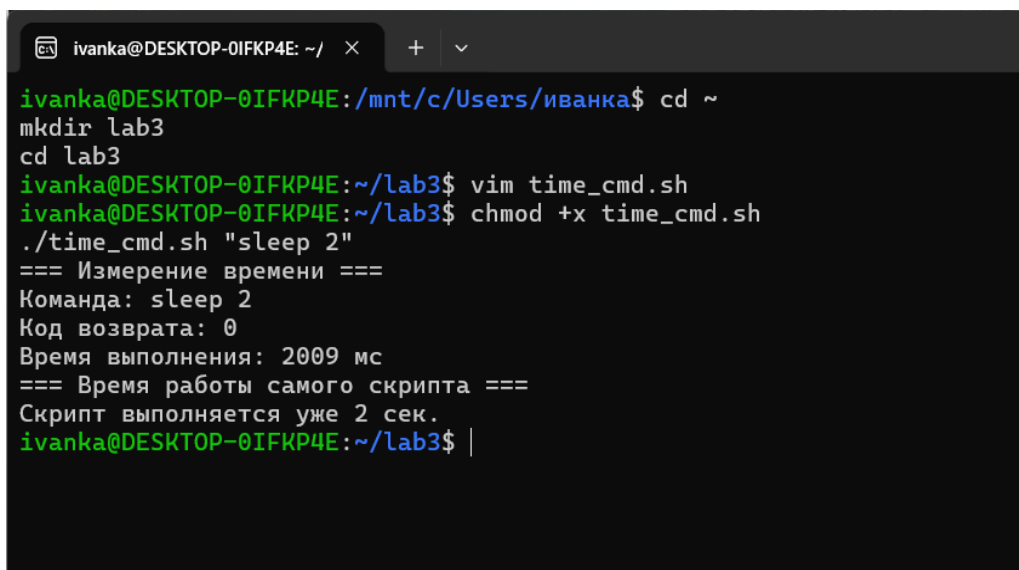
Скрипт:

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Использование: $0 \"команда\""
    exit 1
fi
CMD="$@"
echo "=== Измерение времени ==="
echo "Команда: $CMD"
START=$(python3 -c 'import time; print(int(time.time() * 1000))')
eval "$CMD"
STATUS=$?
END=$(python3 -c 'import time; print(int(time.time() * 1000))')
ELAPSED_MS=$(( END - START ))
echo "Код возврата: $STATUS"
echo "Время выполнения: ${ELAPSED_MS} мс"
echo "=== Время работы самого скрипта ==="
```

```
echo "Скрипт выполняется уже ${SECONDS} сек"
```

Команды запуска скрипта:

```
chmod +x time_cmd.sh  
./time_cmd.sh "sleep 2"
```

A screenshot of a terminal window with a dark background. The window title is 'ivanka@DESKTOP-0IFKP4E: ~/'. The terminal shows the following commands and output:

```
ivanka@DESKTOP-0IFKP4E:/mnt/c/Users/иванка$ cd ~  
mkdir lab3  
cd lab3  
ivanka@DESKTOP-0IFKP4E:~/lab3$ vim time_cmd.sh  
ivanka@DESKTOP-0IFKP4E:~/lab3$ chmod +x time_cmd.sh  
./time_cmd.sh "sleep 2"  
=== Измерение времени ===  
Команда: sleep 2  
Код возврата: 0  
Время выполнения: 2009 мс  
=== Время работы самого скрипта ===  
Скрипт выполняется уже 2 сек.  
ivanka@DESKTOP-0IFKP4E:~/lab3$ |
```

2. Практическая часть

Задание 1. Измерение времени выполнения команды

Был создан скрипт `time_cmd.sh`, адаптированный под macOS. Поскольку стандартная команда `date` на Mac не поддерживает наносекунды (`%N`), для точного расчёта интервала времени в миллисекундах был задействован встроенный вызов Python.

Скрипт:

```
#!/bin/bash  
if [ $# -eq 0 ]; then  
    echo "Использование: $0 \"команда\""
```

```
        exit 1
    fi
    CMD="$@"
    echo "=== Измерение времени ==="
    echo "Команда: $CMD"
    START=$(python3 -c 'import time; print(int(time.time() *
1000))')
    eval "$CMD"
    STATUS=$?
    END=$(python3 -c 'import time; print(int(time.time() *
1000))')
    ELAPSED_MS=$(( END - START ))
    echo "Код возврата: $STATUS"
    echo "Время выполнения: ${ELAPSED_MS} мс"
    echo "=== Время работы самого скрипта ==="
    echo "Скрипт выполняется уже ${SECONDS} сек"
```

Команды запуска скрипта:

```
chmod +x time_cmd.sh
./time_cmd.sh "sleep 2"
```

```
ivanka@DESKTOP-0IFKP4E: ~/ X + v
ivanka@DESKTOP-0IFKP4E:/mnt/c/Users/иванка$ ^C
ivanka@DESKTOP-0IFKP4E:/mnt/c/Users/иванка$ cd ~/lab3
ivanka@DESKTOP-0IFKP4E:~/lab3$ vim gen_file.sh
ivanka@DESKTOP-0IFKP4E:~/lab3$ chmod +x gen_file.sh
ivanka@DESKTOP-0IFKP4E:~/lab3$ ./gen_file.sh rand1.txt &
[1] 840
ivanka@DESKTOP-0IFKP4E:~/lab3$ Генератор запущен. Файл: rand1.txt
PID: 840
cat rand1.txt
11:23:01 1635592449
11:23:11 479118189
11:23:21 2074241897
ivanka@DESKTOP-0IFKP4E:~/lab3$ ./gen_file.sh rand2.txt &
./gen_file.sh rand3.txt &
[2] 893
[3] 900
Генератор запущен. Файл: rand2.txt
PID: 893
ivanka@DESKTOP-0IFKP4E:~/lab3$ Генератор запущен. Файл: rand3.txt
PID: 900
echo "--- rand1 ---"; cat rand1
echo "--- rand2 ---"; cat rand2.txt
echo "--- rand3 ---"; cat rand3.txt
--- rand1 ---
11:23:01 1635592449
11:23:11 479118189
11:23:21 2074241897
11:23:31 1561835957
11:23:41 1957509137
11:23:51 586943805
```

Задание 3. Передача переменной между shell

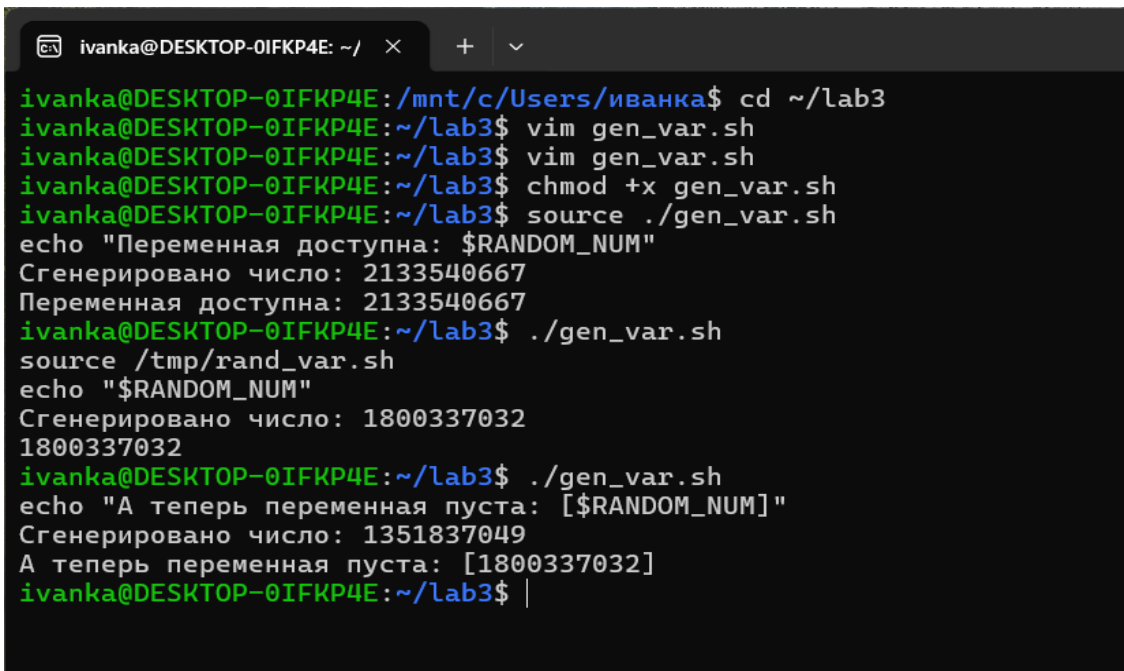
Создан скрипт `gen_var.sh`, который генерирует число, экспортирует переменную `RANDOM_NUM` и формирует файл `/tmp/rand_var.sh`. После запуска исходного скрипта значение проверялось в текущем shell с помощью `source`.

скрипт:

```
#!/bin/bash
K=1103515245
M=2147483648
SEED=$(date +%s)
RANDOM_NUM=$(( (K * SEED) % M ))
export RANDOM_NUM
echo "Сгенерировано число: $RANDOM_NUM"
echo "RANDOM_NUM=$RANDOM_NUM" > /tmp/rand_var.sh
```

Проверка передачи переменной:

```
./gen_var.sh
echo "$RANDOM_NUM"
source /tmp/rand_var.sh
echo "$RANDOM_NUM"
```



```
ivanka@DESKTOP-0IFKP4E: ~/lab3$ cd ~/lab3
ivanka@DESKTOP-0IFKP4E:~/lab3$ vim gen_var.sh
ivanka@DESKTOP-0IFKP4E:~/lab3$ vim gen_var.sh
ivanka@DESKTOP-0IFKP4E:~/lab3$ chmod +x gen_var.sh
ivanka@DESKTOP-0IFKP4E:~/lab3$ source ./gen_var.sh
echo "Переменная доступна: $RANDOM_NUM"
Сгенерировано число: 2133540667
Переменная доступна: 2133540667
ivanka@DESKTOP-0IFKP4E:~/lab3$ ./gen_var.sh
source /tmp/rand_var.sh
echo "$RANDOM_NUM"
Сгенерировано число: 1800337032
1800337032
ivanka@DESKTOP-0IFKP4E:~/lab3$ ./gen_var.sh
echo "А теперь переменная пуста: [$RANDOM_NUM]"
Сгенерировано число: 1351837049
А теперь переменная пуста: [1800337032]
ivanka@DESKTOP-0IFKP4E:~/lab3$ |
```

Задание 4. Решение квадратного уравнения Создан скрипт quad.sh. Он принимает коэффициенты из аргументов командной строки либо запрашивает их интерактивно, проверяет корректность ввода, вычисляет дискриминант и выводит действительные корни. Для вычислений с плавающей точностью используется bc -l.

Скрипт:

```
#!/bin/bash
if [ $# -ge 3 ]; then
    a=$1
    b=$2
    c=$3
else
    read -p "Введите a: " a
    read -p "Введите b: " b
```

```

        read -p "Введите c: " c
    fi
    echo "Уравнение: ${a}x^2 + ${b}x + ${c} = 0"
    if ! [[ "$a" =~ ^-[0-9]+$ && "$b" =~ ^-[0-9]+$ && "$c"
    =~ ^-[0-9]+$ ]]; then
        echo "Ошибка: коэффициенты должны быть целыми числами"
    >&2

```

Лабораторная работа № 3

```

        exit 1
    fi
    if [ "$a" -eq 0 ]; then
        if [ "$b" -eq 0 ]; then
            if [ "$c" -eq 0 ]; then
                echo "Бесконечно много решений"
            else
                echo "Нет решений"
            fi
        else
            echo "Линейное уравнение, корень: $(echo "scale=4;
(-$c)/($b)" | bc)"
        fi
        exit 0
    fi
    D=$((b*b - 4*a*c))
    echo "Дискриминант D = $D"
    if [ "$D" -gt 0 ]; then
        X1=$(echo "scale=4; (-$b + sqrt($D)) / (2*$a)" | bc -
        l)
        X2=$(echo "scale=4; (-$b - sqrt($D)) / (2*$a)" | bc -
        l)
        echo "Два корня: x1=$X1, x2=$X2"
    elif [ "$D" -eq 0 ]; then
        X=$(echo "scale=4; (-$b) / (2*$a)" | bc -l)
        echo "Один корень: x=$X"
    fi

```

```

else
    echo "Действительных корней нет"
fi

```

Проверка для уравнения $x^2 - 3x + 2 = 0$:

```
./quad.sh 1 -3 2
```

```

ivanka@DESKTOP-0IFKP4E: ~/lab3$ vim quad.sh
ivanka@DESKTOP-0IFKP4E: ~/lab3$ vim quad.sh
ivanka@DESKTOP-0IFKP4E: ~/lab3$ ./quad.sh 1 -3 2
Уравнение: 1x^2 + -3x + 2 = 0
Дискриминант D = 1
Два корня: x1=2, x2=1
ivanka@DESKTOP-0IFKP4E: ~/lab3$ ./quad.sh 1 2 1
Уравнение: 1x^2 + 2x + 1 = 0
Дискриминант D = 0
Один корень: x=-1
ivanka@DESKTOP-0IFKP4E: ~/lab3$ ./quad.sh 1 0 1
Уравнение: 1x^2 + 0x + 1 = 0
Дискриминант D = -4
Действительных корней нет
ivanka@DESKTOP-0IFKP4E: ~/lab3$ ./quad.sh 0 2 -4
Уравнение: 0x^2 + 2x + -4 = 0
Линейное уравнение, корень: 2
ivanka@DESKTOP-0IFKP4E: ~/lab3$ ./quad.sh + 1 -5 6
Уравнение: +x^2 + 1x + -5 = 0
Ошибка: коэффициенты должны быть целыми числами
ivanka@DESKTOP-0IFKP4E: ~/lab3$ ./quad.sh +1 -5 6
Уравнение: +1x^2 + -5x + 6 = 0
Ошибка: коэффициенты должны быть целыми числами
ivanka@DESKTOP-0IFKP4E: ~/lab3$ ./quad.sh
Введите a: 1
Введите b: -5
Введите c: 6
Уравнение: 1x^2 + -5x + 6 = 0
Дискриминант D = 1
Два корня: x1=3, x2=2
ivanka@DESKTOP-0IFKP4E: ~/lab3$ |

```

Результат: дискриминант $D = 1$, получены два корня: $x_1 = 2.0000$ и $x_2 = 1.0000$.

Задание 5. Завершение процесса по имени

В mykill.sh реализован режим `-name`. Для поиска процесса используется `pgrep -x`, после чего найденные PID передаются команде `kill`. В качестве проверки был запущен процесс `sleep 1000` и завершён скриптом.

Скрипт:

```

#!/bin/bash
if [ $# -lt 2 ]; then
    echo "Использование:"

```

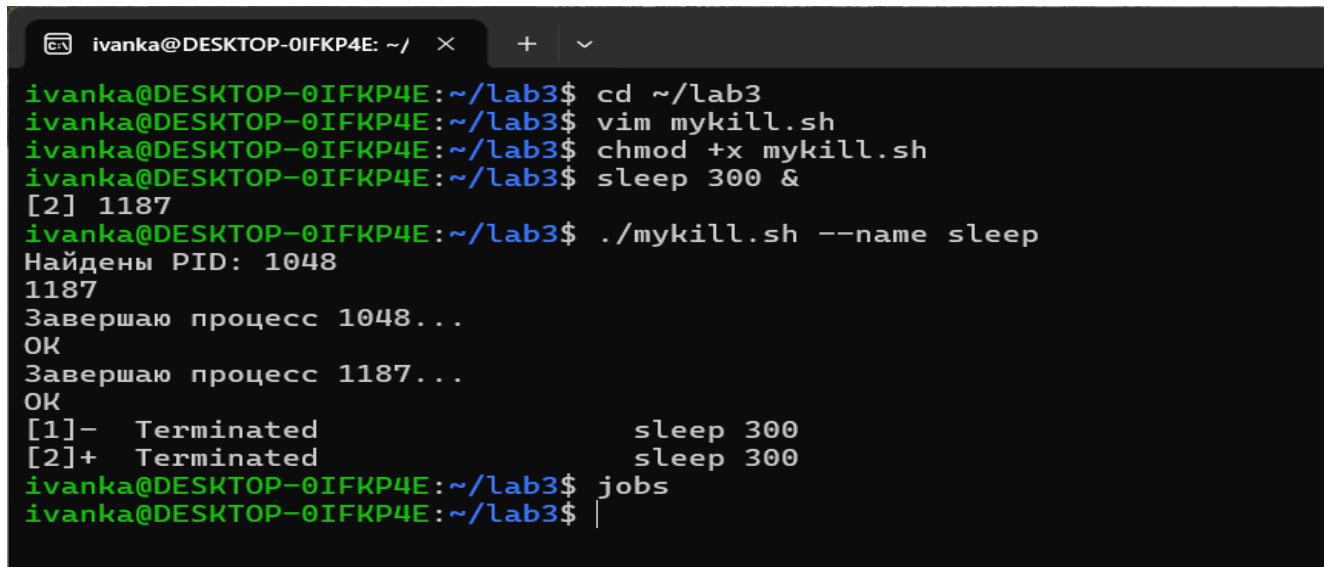
```

    echo " $0 --name <имя_процесса>"
    echo " $0 --user <имя_пользователя>"
    echo " $0 --tty <имя_терминала>"
    exit 1
fi
MODE=$1
VALUE=$2
case "$MODE" in
    --name)
        PIDS=$(pgrep -x "$VALUE")
        ;;
    --user)
        PIDS=$(ps -u "$VALUE" -o pid= | tr -d ' ')
        ;;
    --tty)
        PIDS=$(ps -t "$VALUE" -o pid= | tr -d ' ')
        ;;
    *)
        echo "Неизвестный режим: $MODE" >&2
        exit 1
        ;;
esac
if [ -z "$PIDS" ]; then
    echo "Процессы не найдены"
    exit 1
fi
echo "Найдены PID: $PIDS"
for PID in $PIDS; do
    echo "Завершаю процесс $PID..."
    kill "$PID" 2>/dev/null && echo "OK" || echo "Не
удалось завершить $PID"
done

```

Проверка режима --name:

```
sleep 1000 &  
./mykill.sh --name sleep
```

A terminal window with a dark background and light green text. The window title is 'ivanka@DESKTOP-0IFKP4E: ~/'. The terminal shows a series of commands and their outputs. The user navigates to ~/lab3, edits mykill.sh with vim, sets permissions with chmod, and runs a sleep command in the background. Then, they run ./mykill.sh --name sleep. The script outputs the PID 1048, then 1187, and reports that it is terminating processes 1048 and 1187. Finally, it shows the status of the background jobs as 'Terminated' and 'sleep 300'.

```
ivanka@DESKTOP-0IFKP4E:~/lab3$ cd ~/lab3  
ivanka@DESKTOP-0IFKP4E:~/lab3$ vim mykill.sh  
ivanka@DESKTOP-0IFKP4E:~/lab3$ chmod +x mykill.sh  
ivanka@DESKTOP-0IFKP4E:~/lab3$ sleep 300 &  
[2] 1187  
ivanka@DESKTOP-0IFKP4E:~/lab3$ ./mykill.sh --name sleep  
Найдены PID: 1048  
1187  
Завершаю процесс 1048...  
ОК  
Завершаю процесс 1187...  
ОК  
[1]-  Terminated          sleep 300  
[2]+  Terminated          sleep 300  
ivanka@DESKTOP-0IFKP4E:~/lab3$ jobs  
ivanka@DESKTOP-0IFKP4E:~/lab3$ |
```

Результат: процесс sleep был найден по имени, его PID определён, после чего процесс успешно завершён.

Задание 6. Завершение процессов по пользователю

В mykill.sh реализован режим `-user`. PID процессов пользователя получают командой `ps -u ... -o pid=`, после чего скрипт проходит по найденным PID и отправляет SIGTERM.

Проверка режима `-user`:

```
ps -u prostoi -o pid=  
./mykill.sh --user $USER
```

```
ivanka@DESKTOP-0IFKP4E: ~/ x + v
[2]+ Terminated sleep 300
ivanka@DESKTOP-0IFKP4E:~/lab3$ jobs
ivanka@DESKTOP-0IFKP4E:~/lab3$ sleep 500 &
sleep 600 &
[1] 1234
[2] 1240
ivanka@DESKTOP-0IFKP4E:~/lab3$ ./mykill.sh --user $USER
Найдены PID: 385
388
415
980
1234
1240
1250
1251
1252
1253
Завершаю процесс 385...
OK
Завершаю процесс 388...
OK
Завершаю процесс 415...
OK
Завершаю процесс 980...
OK
Завершаю процесс 1234...
OK
Завершаю процесс 1240...
OK
Завершаю процесс 1250...
```

Задание 7. Завершение процессов по терминалу

В mykill.sh реализован режим `-tty`. Сначала был определён терминал `/dev/pts/0`, затем для `pts/0` были найдены процессы и завершены скриптом.

Определение терминала выполнялось командой:

`tty`

Получен терминал `/dev/pts/0`. Для режима `-tty` использовалось имя `pts/0`.

`./mykill.sh --tty pts/0`

```
ivanka@DESKTOP-0IFKP4E: ~/  × + v
Завершаю процесс 415...
OK
Завершаю процесс 980...
OK
Завершаю процесс 1234...
OK
Завершаю процесс 1240...
OK
Завершаю процесс 1250...
Terminated      ./mykill.sh --user $USER
[1]- Terminated      sleep 500
[2]+ Terminated      sleep 600
ivanka@DESKTOP-0IFKP4E:~/lab3$ tty
/dev/pts/0ivanka@DESKTOP-0IFKP4E:~/lab3$ sleep 700 &
[1] 1291
ivanka@DESKTOP-0IFKP4E:~/lab3$ ./mykill.sh --tty pts/0
Найдены PID: 980
1291
1301
1302
1303
1304
Завершаю процесс 980...
OK
Завершаю процесс 1291...
OK
Завершаю процесс 1301...
Terminated      ./mykill.sh --tty pts/0
[1]+ Terminated      sleep 700
ivanka@DESKTOP-0IFKP4E:~/lab3$ |
```

скрипт нашёл процессы, связанные с pts/0, и завершил их. После завершения процессов терминала появилось сообщение Terminated.

Задание 8. Интерактивная игра «Угадай число»

Создан скрипт game.sh. Он случайно выбирает число от 1 до 100, даёт пользователю до семи попыток, проверяет ввод и сообщает, больше или меньше загаданное число. В ходе проверки число было угадано за 5 попыток.

Скрипт:

```
#!/bin/bash
TARGET=$(( RANDOM % 100 + 1 ))
TRIES=0
MAX=7
echo "=== Игра «Угадай число» ==="
echo "Я загадал число от 1 до 100. У вас $MAX попыток"
while [ "$TRIES" -lt "$MAX" ]; do
    read -p "Ваш вариант: " GUESS
    if ! [[ "$GUESS" =~ ^[0-9]+$ ]]; then
        echo "Введите целое число!"
        continue
```

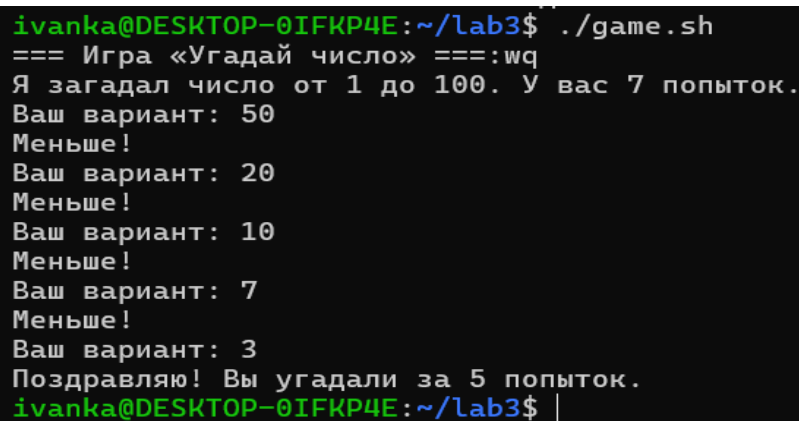
```
fi
TRIES=$(( TRIES + 1 ))
```

Лабораторная работа № 3

```
if [ "$GUESS" -lt "$TARGET" ]; then
    echo "Больше!"
elif [ "$GUESS" -gt "$TARGET" ]; then
    echo "Меньше!"
else
    echo "Поздравляю! Вы угадали за $TRIES попыток."
    exit 0
fi
done
echo "Попытки закончились. Было загадано: $TARGET"
```

Запуск игры:

./game.sh



```
ivanka@DESKTOP-0IFKP4E:~/lab3$ ./game.sh
=== Игра «Угадай число» ===:wq
Я загадал число от 1 до 100. У вас 7 попыток.
Ваш вариант: 50
Меньше!
Ваш вариант: 20
Меньше!
Ваш вариант: 10
Меньше!
Ваш вариант: 7
Меньше!
Ваш вариант: 3
Поздравляю! Вы угадали за 5 попыток.
ivanka@DESKTOP-0IFKP4E:~/lab3$ |
```

Результат: игра корректно принимала варианты пользователя, выводила подсказки «Больше!» и «Меньше!» и завершилась сообщением об успешном угадывании числа за 5 попыток.

11. Результаты работы

1. создан и проверен скрипт time_cmd.sh для измерения времени выполнения команд;

2. создан генератор `gen_file.sh` с записью последовательности чисел в файл и использованием `flock`;

3. создан `gen_var.sh` для генерации переменной и передачи её через `source`;

4. создан `quad.sh` для решения квадратного уравнения;

5. создан `mykill.sh` с режимами поиска процессов по имени, пользователю и терминалу;

6. создан и запущена интерактивная игра `game.sh`;

для всех заданий выполнены практические проверки и сохранены скриншоты результатов.

12. Вывод

В ходе лабораторной работы были освоены более сложные приёмы создания и запуска `shell`-скриптов в `Linux`. Были использованы аргументы командной строки, переменные, циклы и условные конструкции, измерение времени выпол

нения, генерация псевдослучайных чисел, передача переменных между `shell`, блокировка файлов с помощью `flock`, обработка процессов средствами `pgrep`, `ps` и `kill`, а также интерактивный ввод через `stdin`. Практические результаты и скриншоты подтверждают выполнение всех восьми заданий.